HSLU Blockchain Test Environment

Tim Weingärtner

November 1, 2025

Contents

1 Introduction **2**
1.1 Infrastructure Overview . . . 2
1.1.1 Hardhat Node . . . 2
1.1.2 Hardhat Dev . . . 2
1.1.3 SSI Container . . . 2
1.1.4 IPFS Container . . . 3
1.1.5 Web-UI . . . 3
1.2 Usage and Development . . . 3
1.3 Overall Architecture . . . 3

2 Docker Containers and Deployment **4**
2.1 Hardhat Node and Development . . . 4
2.1.1 Prerequisites . . . 4
2.1.2 Building the Docker Images . . . 4
2.1.3 Saving Addresses . . . 4
2.1.4 Accessing the Hardhat Node . . . 5
2.2 Self-Sovereign Identity . . . 5
2.2.1 Prerequisites . . . 5
2.2.2 Building the Docker Images . . . 5
2.2.3 Testing the Setup . . . 5
2.3 InterPlanetary File System . . . 6
2.3.1 Building the Docker Image . . . 6
2.3.2 Starting the IPFS Node . . . 6
2.3.3 Test the IPFS Node . . . 6

3 Running the Examples **6**
3.1 User Interfaces . . . 6
3.2 Run the Webserver . . . 7
3.3 Example 1 - Counter . . . 7
3.4 Example 2 - Crypto Wallet . . . 7
3.5 Example 3 - ERC 20 Token Contract . . . 7
3.6 Example 4 - ERC 721 Token Contract (NFT) . . . 7
3.7 Example 5 - IPFS . . . 7
3.8 Example 6 - SSI Verifiable Credential . . . 8

4 Docker Container Details **8**
4.1 Hardhat Node . . . 8
4.1.1 Dockerfile Overview . . . 8
4.1.2 Hardhat Configuration . . . 8
4.1.3 Package Management . . . 9
4.1.4 Docker Compose Configuration . . . 9

4.2 Hardhat Development . . . 10
4.2.1 Project Structure . . . 10
4.2.2 Dockerfile Overview . . . 10
4.2.3 Hardhat Configuration . . . 11
4.2.4 Smart Contracts Overview . . . 11
4.2.5 Deployment Scripts . . . 11
4.2.6 Test Scripts . . . 14
4.3 Self-Sovereign Identity . . . 16
4.3.1 Dockerfile Overview . . . 17
4.3.2 Docker Compose Configuration . . . 17
4.3.3 Key Generation Script . . . 18
4.3.4 Credential Issuer . . . 19
4.3.5 Test Script . . . 20
4.3.6 Credential Verifier . . . 21
4.3.7 DID Registry . . . 23
4.4 InterPlanetary File System . . . 26
4.4.1 Docker Compose Configuration . . . 26
4.4.2 Entrypoint Script . . . 27
4.5 Oracle Infrastructure . . . 28
4.5.1 Docker Compose Configuration . . . 28
4.6 User Interface . . . 28
4.6.1 JavaScript Functionality . . . 28

1 Introduction

This Blockchain test environment provides a foundational infrastructure designed as an example and a runtime environment for project work and the development of additional use cases. The setup consists of multiple Docker containers, each contributing a specific service to create a fully functional blockchain ecosystem.

1.1 Infrastructure Overview

1.1.1 Hardhat Node

The Hardhat Node serves as the Ethereum blockchain within this environment. It simulates a local Ethereum network where smart contracts can be deployed, tested, and interacted with. This container is crucial for experimenting with smart contracts and understand their behavior on an Ethereum-like network.

1.1.2 Hardhat Dev

Alongside the Hardhat Node, Hardhat Dev automates the deployment of initial smart contracts onto the Ethereum blockchain. It provides tools and scripts that simplify the process of compiling, deploying, and testing smart contracts in this local environment. This setup is particularly useful for quickly setting up a blockchain environment for development and testing purposes.

1.1.3 SSI Container

The Self-Sovereign Identity (SSI) container implements a basic SSI infrastructure, which includes a registry for Decentralized Identifiers (DIDs) and DID Documents. This registry can operate using either a traditional file-based database or the Ethereum blockchain, depending on the use case. The SSI infrastructure allows for the creation, management, and verification of digital identities in a decentralized manner, providing a practical example of how blockchain can be used for identity management.

1.1.4 IPFS Container

The IPFS container offers a local instance of the InterPlanetary File System (IPFS), which is a decentralized storage network. This container enables interaction with IPFS, allowing users to store, retrieve, and manage files in a distributed environment. IPFS is integrated with the blockchain environment to demonstrate how off-chain data can be linked to on-chain assets, such as in the case of storing NFT metadata or digital credentials.

1.1.5 Web-UI

The Web-UI container hosts a set of web pages that serve as the user interface for interacting with the blockchain, SSI infrastructure, and IPFS. These web pages provide examples of how to manage and interact with the various components within the environment, such as deploying and verifying smart contracts, managing digital identities, and uploading files to IPFS. The Web-UI must be run in a web server environment to ensure proper access and functionality. It serves as a practical demonstration of how end-users can interact with the blockchain infrastructure and its associated services.

1.2 Usage and Development

This environment is not only intended for basic exploration but also serves as a flexible platform for developing and testing new blockchain-based use cases. Developers can extend the provided smart contracts, integrate additional services, or modify the existing infrastructure to suit specific project requirements. The modular nature of the Docker containers allows for easy customization and scaling, making this environment a valuable tool for both learning and innovation in blockchain technology.

Unfortunately, Hardhat at the moment does not allow to persist the blockchain. This means that once you stopped the Hardhat container, you have to deploy your contracts again. This will also change their addresses!

1.3 Overall Architecture

The following diagram provides a visual overview of the Blockchain test environment. This architecture demonstrates how the various Docker containers and services interact with each other to create a functional blockchain ecosystem.

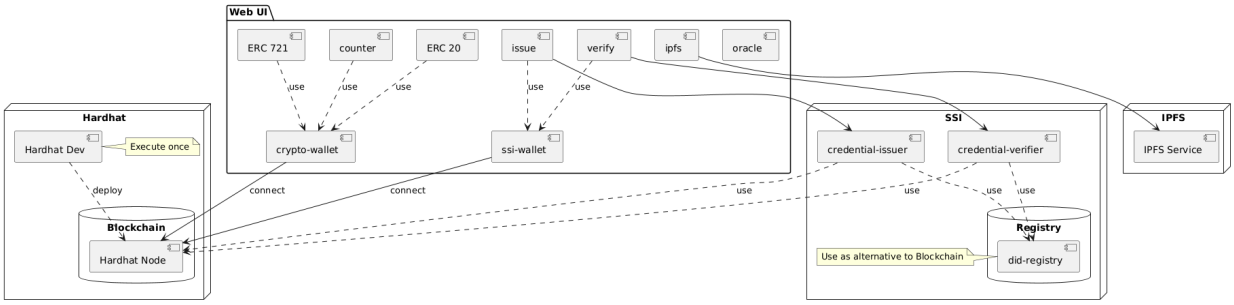

Figure 1: Architecture of the Blockchain Test Environment

This Blockchain test environment includes the following components:

- Hardhat Node: Provides an Ethereum-like blockchain environment for deploying and testing smart contracts.
- Hardhat Dev: Automates the deployment of smart contracts to the Hardhat Node.
- SSI Container: Implements a Self-Sovereign Identity (SSI) infrastructure, including a DID registry.
- IPFS Container: Provides a local IPFS service for decentralized file storage and retrieval.

- Web UI: The front-end interface for interacting with the blockchain, SSI, and IPFS services.

The arrows in the diagram indicate the interactions and data flow between these components, illustrating how they work together to support blockchain and SSI functionalities.

2 Docker Containers and Deployment

This section provides step-by-step instructions for setting up and deploying the infrastructure using Docker. The deployment process involves the use of Docker and Docker Compose to manage and run the environment.

2.1 Hardhat Node and Development

The setup includes a Hardhat development environment configured for local blockchain development and testing.

2.1.1 Prerequisites

Ensure that Docker and Docker Compose are installed on your system.

2.1.2 Building the Docker Images

Create a hardhat network:

```
docker network create --driver bridge hardhat-network
```

Navigate to the Hardhat directory containing the docker-compose.yml files. Run the following command to build the Docker images:

```
docker compose build
```

Running the Containers To start the Hardhat node and development environment, execute:

```
docker compose up -d
```

This command will start the Hardhat node and execute the development workflow, which includes installing dependencies, compiling contracts, deploying them. The Hardhat development will only be executed once and will terminate after the contracts are deploys.

The script automatically run several test cases at the end. Ensure that all tests are correct:

```
 6 passing (1s)
```

Unfortunately, Hardhat at the moment does not allow to persist the blockchain.

If you have stopped the hardhat container you have to run again the deployment (hardhat-dev). This means that also the addresses of the smart contracts change and you have to also update the ssi container.

2.1.3 Saving Addresses

Make sure you copy the account addresses and the addresses of the deployed Smart Contracts from the outputs:

```
Account 0: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944 bacb478cbed5efcae784d7bf4f2ff80

...
```

```
Counter deployed to: 0x5FbDB2315678afecb367f032d93F642f64180aa3

MyToken deployed to: 0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512

MyNFT deployed to: 0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0

DIDRegisstry deployed to: 0xCf7Ed3AccA5a467e9e704C703E8D87F634fB0Fc9
```

2.1.4 Accessing the Hardhat Node

The Hardhat node will be available on port 8545. You can interact with it using tools like `Remix`. You can access the Remix Ethereum IDE by visiting the following link: Remix Ethereum IDE.

2.2 Self-Sovereign Identity

The Self-Sovereign Identity (SSI) infrastructure is designed to simulate the key components of an SSI system, including an Credential Issuer, a Credential Verifier, and a DID Registry. This infrastructure allows for the generation, issuance, and verification of digital credentials using decentralized identifiers (DIDs).

2.2.1 Prerequisites

Ensure that Docker and Docker Compose are installed on your system.

Ensure that you have installed the Hardhat container.

Ensure that the did-registry contract address in $did-registry/index.js$ (Line 1) and $did-registry/server.js$ (Line 1) is correct or update it.

```
const contractAddress = "0xCf7Ed3AccA5a467e9e704C703E8D87F634fB0Fc9";
```

```
const registryAddress = '0xCf7Ed3AccA5a467e9e704C703E8D87F634fB0Fc9';
```

Ensure that the account address in $did-registry/index.js$ (Line 102) is correct or update it. This should not change.

```
account = web3.eth.accounts.privateKeyToAccount('0
    xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5 efcae784d7bf4f2ff80');
// Using a pre-defined blockchain account
web3.eth.accounts.wallet.add(account);
```

2.2.2 Building the Docker Images

Navigate to the SSI directory containing the docker-compose.yml. Run the following commands to build and start the Docker containers:

```
docker compose build
docker compose up -d
```

2.2.3 Testing the Setup

To test the SSI setup, run the test script:

```
node test.js
```

2.3 InterPlanetary File System

This IPFS (InterPlanetary File System) Docker container allows the network to store, retrieve, and manage files in a distributed manner, supporting the other services.

To deploy the IPFS infrastructure, follow these steps:

2.3.1 Building the Docker Image

Navigate to the IPFS directory containing the Dockerfile and the `docker-compose.yml` file. Get a pre-build ipfs docker with:

```
docker compose pull ipfs
```

2.3.2 Starting the IPFS Node

Start the IPFS node by running:

```
docker compose up -d
```

This command starts the IPFS service in detached mode, making it accessible to other containers in the network.

2.3.3 Test the IPFS Node

You can interact with the IPFS node using its API on port 5001 or through the gateway on port 8080. For example, you can add files to IPFS or retrieve them via HTTP.

To test you can run the testfile:

```
node store_and_get_file.js
```

This will store the *file.txt* in IPFS and the retrieve it to *ouput.txt*.

3 Running the Examples

3.1 User Interfaces

The user interface is in the web-ui directory and is composed of the following HTML pages:

- **counter.html**: A page for interacting with a counter contract deployed on the blockchain.
- **crypto-wallet.html**: A page that serves as a user interface for managing crypto wallets and performing Ether transfers.
- **ERC20.html**: A page dedicated to interacting with an ERC20 token contract.
- **ERC721.html**: A page for creating, viewing, and transferring NFTs using an ERC721 contract.
- **ipfs.html**: A page for uploading and downloading files from IPFS.
- **ssi-wallet.html**: A page that acts as a wallet for managing digital credentials.
- **issue.html**: A page for issuing digital credentials.
- **verify.html**: A page for verifying the authenticity of issued credentials.

3.2 Run the Webserver

You can either run a Webserver in Visual Studio Code (Live Server) or any other Webserver (e.g. python3):

```
python3 -m http.server 5500
```

You should now be able to access the pages via

```
http://localhost:5500/
```

Since Counter, ERC20 and ERC721 connect to Smart Contracts you must change the correct addresses in the Javascript files of these examples. See 2.1.3

Change the contract address in the first line. For Counter in: $web - ui/js/counter.js$

For MyToken (ERC20) in: $web - ui/js/ERC20.js$

For MyNFT (ERC721) in: $web - ui/js/ERC721.js$

3.3 Example 1 - Counter

The Counter page shows the *CurrentCount* and provides buttons to increment and decrement the counter. If not done, you must first select an Ethereum address with ETH to use the counter with. You can change this address with the *ChangeAddress* Button.

Only incrementing and decrementing cost Gas. The counter can not be lower than 0.

Tipp: Use the console log if it does not work and make sure you have changed the Smart Contract address. In this case you might have to reload the page with $Shift - reload$.

3.4 Example 2 - Crypto Wallet

The Crypto Wallet shows you a set of User addresses. For each address you can display the private key and the actual balance.

You can also transfer ETH from one address to another.

The *DeleteAddress* only deletes the address from this page and does not delete it on the Blockchain.

3.5 Example 3 - ERC 20 Token Contract

The ERC 20 Token page allows you first to see your actual amount of tokens (balance). You might have to select an user address first (*ChangeAddresse*). In the beginning all tokens are with the first address.

You then can transfer tokens to other addresses and review the transfer by changing the user address.

3.6 Example 4 - ERC 721 Token Contract (NFT)

The ERC 721 Token page allows you to mint, view and transfer NFTs. You can select any text in the Metadata URL field. This could be a sample webaddress (e.g. http://www.hslu.ch). An alert box pops up if everything was ok.

Next you can view the Metadata URL for any minted NFT. The first NFT is minted to NFT-ID 0. The next is 1 and so on.

Finally you can transfer a NFT to another address. This changes the ownership of this NFT.

3.7 Example 5 - IPFS

The IPFS page allows you to upload and download a document from IPFS. If you upload a document the CID (Content ID) of the document is displayed. You need this CID to download a file which is then saved automatically in your download folder.

3.8 Example 6 - SSI Verifiable Credential

The SSI Issue page allows you to issue Verifiable Credentials (VCs) on either a registry database or the blockchain. You can select the registry and one out of two issuers. The DID either has to start with $did : db :$ for the database and $did : eth :$ for the blockchain. In the case of the blockchain the DID has to be a valid Ethereum address like $did : eth : 0x8626f6940E2eb28930eFb4CeF49B2d1F2C9C1199$.

Once you issued a credential this VC is displayed in the text box and is stored in the local database of your browser. You can see all stored VC with the SSI Wallet. Here you can also delete VCs out of your local storage. The VC is not deleted on the registry.

The SSI Verify page allows you to verify VC out of your wallet. You can select a VC out of the list and can press verify. If you edit fields the verification will fail.

4 Docker Container Details

4.1 Hardhat Node

The project consists of the following key files:

- **Dockerfile**: Defines the Docker image for the Hardhat environment.
- **hardhat.config.js**: The configuration file for the Hardhat project, specifying network settings and Solidity version.
- **package.json**: Manages the project's dependencies, including the Hardhat package.
- **docker-compose.yml**: Defines the services, networks, and volumes required to run the Hardhat infrastructure using Docker Compose.

4.1.1 Dockerfile Overview

The Dockerfile sets up the environment for running Hardhat. It is assumed to be located in the 'hardhat-node' directory.

Listing 1: Dockerfile for Hardhat Node

```
# Sample Dockerfile for setting up a Hardhat environment
FROM node:16

WORKDIR /usr/src/app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 8545

CMD ["npx", "hardhat", "node"]
```

4.1.2 Hardhat Configuration

The `hardhat.config.js` file configures the Hardhat network and sets the Solidity compiler version.

Listing 2: hardhat.config.js

```
module.exports = {
    networks: {
        hardhat: {
```

```
            chainId: 1337,
            accounts: {
                mnemonic: "test test test test test test test test test test test junk
                    "
            }
        }
    },
    solidity: "0.8.18",
};
```

4.1.3 Package Management

The `package.json` file defines the project's dependencies, including Hardhat. This file ensures that the correct version of Hardhat is installed when the Docker container is built.

Listing 3: package.json

```
{
    "name": "hardhat-node",
    "version": "1.0.0",
    "dependencies": {
        "hardhat": "^2.12.4"
    }
}
```

4.1.4 Docker Compose Configuration

The `docker-compose.yml` file orchestrates the services required for the Hardhat environment. It defines two services:

- **hardhat-node**: Runs the Hardhat node, exposing it on port 8545.
- **hardhat-dev**: Handles the development environment, including compiling contracts and running tests.

Listing 4: docker-compose.yml

```
services:
  hardhat-node:
    build: ./hardhat-node
    ports:
      - "8545:8545"
    networks:
      - hardhat-network

  hardhat-dev:
    build: ./hardhat-dev
    depends_on:
      - hardhat-node
    volumes:
      - ./hardhat-dev:/usr/src/app
      - shared-data:/usr/src/app/shared
    networks:
      - hardhat-network
    command: sh -c "npm install && npm run compile && sleep 5 && npm run deploy_all &&
        npm run test"

networks:
  hardhat-network:
```

```
    external: true

volumes:
  shared-data:
```

4.2 Hardhat Development

The primary purpose of the Hardhat development is to deploy the necessary smart contracts that other projects will depend on. The deployment is executed once, after which the deployed contracts are available for use in various applications.

4.2.1 Project Structure

The project consists of the following key files and contracts:

- **Dockerfile**: Defines the Docker image for the Hardhat environment.
- **hardhat.config.js**: Configuration file for the Hardhat project, specifying network settings and Solidity version.
- **Smart Contracts**:
  - **Oracle.sol**: Solidity contract for handling oracle-related functionality.
  - **MyToken.sol**: Solidity contract for an ERC-20 token.
  - **Counter.sol**: Solidity contract implementing a simple counter.
  - **MyNFT.sol**: Solidity contract for an ERC-721 Non-Fungible Token (NFT).
  - **DIDRegistry.sol**: Solidity contract for a Decentralized Identifier (DID) registry.
- **Deployment Scripts**:
  - **deploy_my_token.js**: Deploys the MyToken contract.
  - **deploy_my_nft.js**: Deploys the MyNFT contract.
  - **deploy_counter.js**: Deploys the Counter contract.
  - **deploy_did_registry.js**: Deploys the DIDRegistry contract using ‘create2‘.
  - **deploy_all.js**: A script to deploy all contracts at once.
- **Test Scripts**:
  - **MyToken.js**: Test cases for the MyToken contract.
  - **MyNFT.js**: Test cases for the MyNFT contract.
  - **Counter.js**: Test cases for the Counter contract.
  - **DIDRegistry.js**: Test cases for the DIDRegistry contract.

4.2.2 Dockerfile Overview

The Dockerfile sets up the environment for running Hardhat and deploying the smart contracts. It installs necessary dependencies and prepares the environment.

Listing 5: Dockerfile for Hardhat Development Environment

```
# Dockerfile to set up Hardhat development environment
FROM node:16

WORKDIR /usr/src/app
```

```
COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 8545

CMD ["npx", "hardhat", "node"]
```

4.2.3 Hardhat Configuration

The `hardhat.config.js` file configures the Hardhat network and specifies the Solidity compiler version. This setup allows contracts to be deployed to a local Hardhat network.

Listing 6: hardhat.config.js

```
require("@nomiclabs/hardhat-ethers");
require("@nomicfoundation/hardhat-chai-matchers");

module.exports = {
  defaultNetwork: "localhost",
  networks: {
    hardhat: {},
    localhost: {
      url: "http://hardhat-node:8545",
      accounts: {
        mnemonic: "test test test test test test test test test test test junk"
      }
    }
  },
  solidity: "0.8.20",
};
```

4.2.4 Smart Contracts Overview

The following contracts are included in the project:

- **MyToken.sol**: Implements an ERC-20 token with basic functionalities such as minting and transferring tokens.
- **Counter.sol**: A simple contract that increments and decrements a counter.
- **MyNFT.sol**: Implements an ERC-721 NFT with basic minting capabilities.
- **DIDRegistry.sol**: Handles registration and management of Decentralized Identifiers (DIDs).

4.2.5 Deployment Scripts

The following scripts are used to deploy the smart contracts:

deploy_my_token.js Deploys the MyToken contract:

Listing 7: deploy_my_token.js

```
async function main() {
    const MyToken = await ethers.getContractFactory("MyToken");
    const myToken = await MyToken.deploy();
    await myToken.deployed();
```

```
    console.log("MyToken deployed to:", myToken.address);
}

main()
    .then(() => process.exit(0))
    .catch((error) => {
        console.error(error);
        process.exit(1);
    });
```

deploy_my_nft.js Deploys the MyNFT contract:

Listing 8: deploy_my_nft.js

```
async function main() {
    const MyNFT = await ethers.getContractFactory("MyNFT");
    const myNFT = await MyNFT.deploy();
    await myNFT.deployed();
    console.log("MyNFT deployed to:", myNFT.address);
}

main()
    .then(() => process.exit(0))
    .catch((error) => {
        console.error(error);
        process.exit(1);
    });
```

deploy_counter.js Deploys the Counter contract:

Listing 9: deploy_counter.js

```
async function main() {
    const Counter = await ethers.getContractFactory("Counter");
    const counter = await Counter.deploy();
    await counter.deployed();
    console.log("Counter deployed to:", counter.address);
}

main()
    .then(() => process.exit(0))
    .catch((error) => {
        console.error(error);
        process.exit(1);
    });
```

deploy_did_registry.js Deploys the DIDRegistry contract using the ‘create2‘ opcode for deterministic contract deployment:

Listing 10: deploy_did_registry.js

```
const { ethers } = require("hardhat");
const { keccak256 } = require("ethers/lib/utils");

async function main() {
  const [deployer] = await ethers.getSigners();
  const factory = await ethers.getContractFactory("DIDRegistry");
```

```
  // Create the initialization code for the contract
  const bytecode = factory.bytecode;
  const constructorTypes = ["address"];
  const constructorArgs = [deployer.address];
  const initCode = bytecode + ethers.utils.defaultAbiCoder.encode(constructorTypes,
      constructorArgs).slice(2);

  // Define a salt for deterministic address generation
  const salt = ethers.utils.keccak256(ethers.utils.toUtf8Bytes("DIDRegistrySalt"));

  // Calculate the deterministic address
  const create2Address = ethers.utils.getCreate2Address(deployer.address, salt,
      keccak256(initCode));
  console.log(`Expected DIDRegistry address: ${create2Address}`);

  // Deploy the contract using create2
  const tx = await deployer.sendTransaction({
    from: deployer.address,
    data: initCode,
    gasLimit: 1000000,
    nonce: await deployer.getTransactionCount(),
  });

  await tx.wait();

  console.log("DIDRegistry deployed to:", create2Address);
}

main()
  .then(() => process.exit(0))
  .catch((error) => {
    console.error(error);
    process.exit(1);
  });
```

deploy_all.js Deploys all contracts (Counter, MyToken, MyNFT, DIDRegistry, and Oracle) at once:

Listing 11: deploy_all.js

```
const { ethers } = require("hardhat");

async function main() {
    // Deploy Counter
    const Counter = await ethers.getContractFactory("Counter");
    const counter = await Counter.deploy();
    await counter.deployed();
    console.log("Counter deployed to:", counter.address);

    // Deploy MyToken
    const MyToken = await ethers.getContractFactory("MyToken");
    const myToken = await MyToken.deploy();
    await myToken.deployed();
    console.log("MyToken deployed to:", myToken.address);

    // Deploy MyNFT
    const MyNFT = await ethers.getContractFactory("MyNFT");
    const myNFT = await MyNFT.deploy();
    await myNFT.deployed();
    console.log("MyNFT deployed to:", myNFT.address);
```

```
    // Deploy DIDRegistry
    const DIDRegistry = await ethers.getContractFactory("DIDRegistry");
    const didRegistry = await DIDRegistry.deploy();
    await didRegistry.deployed();
    console.log("DIDRegistry deployed to:", didRegistry.address);
}

main()
    .then(() => process.exit(0))
    .catch((error) => {
    console.error(error);
    process.exit(1);
    });
```

4.2.6 Test Scripts

The following test scripts are used to validate the functionality of the deployed contracts:

MyToken.js Tests the functionality of the MyToken contract:

Listing 12: MyToken.js

```
// Test cases for MyToken contract
const { expect } = require("chai");
const { ethers } = require("hardhat");

describe("MyToken", function () {
  let myToken;

  beforeEach(async function () {
    const MyToken = await ethers.getContractFactory("MyToken");
    myToken = await MyToken.deploy();
    await myToken.deployed();
  });

  it("should have correct name and symbol", async function () {
    expect(await myToken.name()).to.equal("MyToken");
    expect(await myToken.symbol()).to.equal("MTK");
  });

  it("should mint tokens", async function () {
    const [owner] = await ethers.getSigners();
    await myToken.mint(owner.address, 100);
    expect(await myToken.balanceOf(owner.address)).to.equal(100);
  });

  it("should burn tokens", async function () {
    const [owner] = await ethers.getSigners();
    await myToken.mint(owner.address, 100);
    await myToken.burn(owner.address, 50);
    expect(await myToken.balanceOf(owner.address)).to.equal(50);
  });
});
```

MyNFT.js Tests the functionality of the MyNFT contract:

Listing 13: MyNFT.js

```
// Test cases for MyNFT contract
const { expect } = require("chai");
const { ethers } = require("hardhat");

describe("MyNFT", function () {
  let myNFT;
  let owner;
  let addr1;

  beforeEach(async function () {
    [owner, addr1] = await ethers.getSigners();
    const MyNFT = await ethers.getContractFactory("MyNFT");
    myNFT = await MyNFT.deploy(owner.address);
    await myNFT.deployed();
  });

  it("should have correct name and symbol", async function () {
    expect(await myNFT.name()).to.equal("MyNFT");
    expect(await myNFT.symbol()).to.equal("NFT");
  });

  it("should mint an NFT", async function () {
    const tokenURI = "http://test.hslu.ch/1";
    await myNFT.safeMint(addr1.address, tokenURI);
    expect(await myNFT.ownerOf(0)).to.equal(addr1.address);
    expect(await myNFT.tokenURI(0)).to.equal(tokenURI);
  });

  it("should burn an NFT", async function () {
    const tokenURI = "http://test.hslu.ch/1";
    await myNFT.safeMint(addr1.address, tokenURI);
    await myNFT.burn(0);
    await expect(myNFT.ownerOf(0)).to.be.revertedWith("ERC721: owner query for
        nonexistent token");
  });
});
```

Counter.js Tests the functionality of the Counter contract:

Listing 14: Counter.js

```
// Test cases for Counter contract
const { expect } = require("chai");
const { ethers } = require("hardhat");

describe("Counter", function () {
  let counter;

  beforeEach(async function () {
    const Counter = await ethers.getContractFactory("Counter");
    counter = await Counter.deploy();
    await counter.deployed();
  });

  it("should start with a count of 0", async function () {
    expect(await counter.getCounter()).to.equal(ethers.BigNumber.from(0));
  });
```

```
  it("should increment the count", async function () {
    await counter.increment();
    expect(await counter.getCounter()).to.equal(ethers.BigNumber.from(1));
  });

  it("should decrement the count", async function () {
    await counter.increment();
    await counter.decrement();
    expect(await counter.getCounter()).to.equal(ethers.BigNumber.from(0));
  });

  it("should not decrement below 0", async function () {
    await expect(counter.decrement()).to.be.revertedWith("Counter: count must be
        greater than zero");
  });
});
```

DIDRegistry.js Tests the functionality of the DIDRegistry contract:

Listing 15: DIDRegistry.js

```
// Test cases for DIDRegistry contract
const { expect } = require("chai");
const { ethers } = require("hardhat");

describe("DIDRegistry", function () {
  let didRegistry;
  let owner;
  let addr1;

  beforeEach(async function () {
    [owner, addr1] = await ethers.getSigners();
    const DIDRegistry = await ethers.getContractFactory("DIDRegistry");
    didRegistry = await DIDRegistry.deploy();
    await didRegistry.deployed();
  });

  it("should set a DID", async function () {
    const address = "did:test:123";
    const didDocument = "https://example.com/did/1";
    await didRegistry.setDID(addr1.address, didDocument);
    expect(await didRegistry.getDID(addr1.address)).to.equal(didDocument);
  });

  it("should delete a DID", async function () {
    const address = "did:test:123";
    const didDocument = "https://example.com/did/1";
    await didRegistry.setDID(addr1.address, didDocument);
    await didRegistry.deleteDID(addr1.address);
    expect(await didRegistry.getDID(addr1.address)).to.equal("");
  });
});
```

4.3 Self-Sovereign Identity

This section provides detailed instructions and an overview of a Docker-based Self-Sovereign Identity (SSI) infrastructure. The SSI environment is designed to simulate the key components of an SSI system, including

an Credential Issuer, a Credential Verifier, and a DID Registry. This infrastructure allows for the generation, issuance, and verification of digital credentials using decentralized identifiers (DIDs).

The project consists of the following key components:

- **key-generator**: Generates public-private key pairs used during credential issuance.
- **credential-issuer**: Issues digital credentials in JSON format, signing them with the issuer's private key.
- **credential-verifier**: Verifies the authenticity of issued credentials by checking signatures against the DID Registry.
- **did-registry**: Maintains a registry of DIDs and their corresponding documents.

4.3.1 Dockerfile Overview

The Dockerfile sets up the environment for the key generation and credential issuance services. The following Dockerfile is used:

Listing 16: Dockerfile for Credential Issuer and Key Generation Services

```
# Dockerfile for setting up the SSI components
FROM node:16

WORKDIR /usr/src/app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 9001 9002 9003

CMD ["node", "issuer.js"]
```

4.3.2 Docker Compose Configuration

The `docker-compose.yml` file orchestrates the services required for the SSI infrastructure. It defines the following services:

- **key-generator**: Generates key pairs for the issuers and shares them via a volume.
- **credential-issuer**: Issues credentials, signs them, and registers them in the DID Registry.
- **credential-verifier**: Verifies credentials against the DID Registry.
- **did-registry**: Manages the storage and retrieval of DID documents.

Listing 17: docker-compose.yml

```
services:
  key-generator:
    build: ./key-generator
    networks:
      - hardhat-network
    volumes:
      - keys:/keys
    command: ["sh", "-c", "node generate-keys.js && cp -r ./keys/* /keys/"]
```

```
  credential-issuer:
    build: ./credential-issuer
    ports:
      - "9001:9001"
    networks:
      - hardhat-network
    volumes:
      - keys:/usr/src/app/keys
    depends_on:
      - key-generator

  credential-verifier:
    build: ./credential-verifier
    ports:
      - "9002:9002"
    networks:
      - hardhat-network

  did-registry:
    build: ./did-registry
    ports:
      - "9003:9003"
    networks:
      - ssi-network
      - hardhat-network
    volumes:
      - shared-data:/usr/src/app/shared

networks:
  hardhat-network:
    external: true

volumes:
  keys:
  shared-data:
```

4.3.3 Key Generation Script

The `generate-keys.js` script is responsible for generating public-private key pairs for the issuers:

Listing 18: generate-keys.js

```
const crypto = require('crypto');
const fs = require('fs');
const path = require('path');

// Function to generate and save key pair
const generateKeyPair = (issuerName) => {
  const { publicKey, privateKey } = crypto.generateKeyPairSync('rsa', {
    modulusLength: 2048,
  });

  fs.writeFileSync(path.join(__dirname, `./keys/${issuerName}_public.pem`), publicKey.
      export({ type: 'pkcs1', format: 'pem' }));
  fs.writeFileSync(path.join(__dirname, `./keys/${issuerName}_private.pem`),
      privateKey.export({ type: 'pkcs1', format: 'pem' }));
};
```

```
// Create keys directory if it doesn't exist
const keysDir = path.join(__dirname, 'keys');
if (!fs.existsSync(keysDir)) {
  fs.mkdirSync(keysDir);
}

// Generate key pairs for two issuers
generateKeyPair('issuer1');
generateKeyPair('issuer2');

console.log('Keys generated and saved to files.');
```

4.3.4 Credential Issuer

The `issuer.js` script implements the credential issuer component. It generates credentials, signs them, and registers them in the DID Registry:

Listing 19: issuer.js

```
const express = require('express');
const cors = require('cors');
const crypto = require('crypto');
const fs = require('fs');
const axios = require('axios');
const app = express();
const port = 9001;

app.use(cors());
app.use(express.json());

app.post('/issue-credential', async (req, res) => {
    const { issuer, did, credential, registry } = req.body;
    console.log(`Received request to issue credential for DID: ${did}, Issuer: ${
        issuer}, Registry: ${registry}`);

    try {
        // Load issuer's private key
        const privateKey = fs.readFileSync(`./keys/${issuer}_private.pem`, 'utf8');
        console.log(`Loaded private key for issuer: ${issuer}`);

        // Create DID Document
        const didDocument = {
            id: did,
            publicKey: fs.readFileSync(`./keys/${issuer}_public.pem`, 'utf8'),
            authentication: "exampleAuthentication",
            service: [],
            issuer
        };
        console.log('Created DID document:', didDocument);

        let registryResponse;
        if (registry === 'db') {
            // Register the DID Document in the local DID Registry
            registryResponse = await axios.post('http://did-registry:9003/dids/db',
                didDocument);
        } else if (registry === 'blockchain') {
            // Register the DID Document in the blockchain DID Registry
            registryResponse = await axios.post('http://did-registry:9003/dids/
                blockchain', didDocument);
```

```
        }
        console.log('Registered DID document:', registryResponse.data);

        // Simulate issuing a credential
        const issuedCredential = {
            content: credential,
            issuedTo: did,
            issuedAt: new Date().toISOString(),
            issuer: issuer,
            registry: registry // Include registry information
        };

        // Log the exact data being hashed
        const jsonString = JSON.stringify(issuedCredential);
        console.log('JSON string being hashed:', jsonString);

        // Create a hash of the credential
        const documentHash = crypto.createHash('sha256').update(jsonString).digest('
            hex');
        console.log(`Created document hash: ${documentHash}`);

        // Sign the hash with issuer's private key
        const sign = crypto.createSign('SHA256');
        sign.update(documentHash);
        sign.end();
        const signature = sign.sign(privateKey, 'base64');
        console.log(`Created signature: ${signature}`);

        // Verify signature on issuer side (for validation)
        const verify = crypto.createVerify('SHA256');
        verify.update(documentHash);
        verify.end();
        const isValidSignature = verify.verify(didDocument.publicKey, signature, '
            base64');
        console.log(`Signature valid on issuer side: ${isValidSignature}`);

        issuedCredential.signature = signature;
        res.json(issuedCredential);
    } catch (error) {
        console.error('Error issuing credential:', error);
        res.status(500).json({ error: 'Error issuing credential' });
    }
});

app.listen(port, () => {
    console.log(`Credential issuer running at http://localhost:${port}`);
});
```

4.3.5 Test Script

The `test.js` script tests the functionality of the SSI components by creating a DID Document, signing it, and verifying the signature:

Listing 20: test.js

```
const crypto = require('crypto');
const fs = require('fs');

// Load the private key for signing
```

```
const privateKey = fs.readFileSync('./keys/issuer1_private.pem', 'utf8');
// Load the public key for verification
const publicKey = fs.readFileSync('./keys/issuer1_public.pem', 'utf8');

// Create a sample DID document
const didDocument = {
  id: 'did:example:1234',
  publicKey: publicKey,
  authentication: "exampleAuthentication",
  service: [],
  issuer: 'issuer1'
};

// Create a hash of the DID Document excluding the signature field
const documentHash = crypto.createHash('sha256').update(JSON.stringify(didDocument)).
    digest('hex');
console.log(`Created document hash: ${documentHash}`);

// Sign the hash
const sign = crypto.createSign('SHA256');
sign.update(documentHash);
sign.end();
const signature = sign.sign(privateKey, 'hex');
console.log(`Created signature: ${signature}`);

// Verify the signature
const verify = crypto.createVerify('SHA256');
verify.update(documentHash);
verify.end();
const isValidSignature = verify.verify(publicKey, Buffer.from(signature, 'hex'));
console.log(`Signature valid: ${isValidSignature}`);
```

4.3.6 Credential Verifier

The `verifier.js` script implements the credential verifier component. It is responsible for verifying the authenticity of issued credentials by checking their signatures against the public keys stored in the DID Registry. The verifier queries the DID Registry, retrieves the associated public key, and uses it to verify the digital signature of the credential.

Credential Verifier Implementation The credential verifier is implemented as an Express.js service. It listens for verification requests on port 9002 and processes the incoming credentials to validate their authenticity.

Listing 21: verifier.js

```
const express = require('express');
const cors = require('cors');
const axios = require('axios');
const crypto = require('crypto');
const app = express();
const port = 9002;

app.use(cors());
app.use(express.json());

app.post('/verify-credential', async (req, res) => {
const { credential } = req.body;
console.log('Received verification request for:', credential);
```

```
javascript
Code kopieren
try {
    console.log(`Querying DID registry for DID: ${credential.issuedTo}`);
    let response;
    if (credential.registry === 'db') {
        response = await axios.get(`http://did-registry:9003/dids/db/${credential.
            issuedTo}`);
    } else if (credential.registry === 'blockchain') {
        response = await axios.get(`http://did-registry:9003/dids/blockchain/${
            credential.issuedTo}`);
    }
    const didDocument = response.data;
    console.log('DID document found:', didDocument);

    if (!credential.signature) {
        throw new Error('Credential does not contain a signature');
    }

    if (!didDocument.issuer || !didDocument.publicKey) {
        throw new Error('Issuer or publicKey is not defined in the DID document');
    }

    console.log(`Using issuer: ${didDocument.issuer} and publicKey: ${didDocument.
        publicKey}`);

    // Log the exact data being hashed
    const { signature, ...credentialWithoutSignature } = credential;
    const jsonString = JSON.stringify(credentialWithoutSignature);
    console.log('JSON string being hashed:', jsonString);

    const documentHash = crypto.createHash('sha256').update(jsonString).digest('hex');
    console.log(`Created document hash: ${documentHash}`);

    const verify = crypto.createVerify('SHA256');
    verify.update(documentHash);
    verify.end();
    const isValidSignature = verify.verify(didDocument.publicKey, Buffer.from(
        signature, 'base64'));
    console.log('Signature valid:', isValidSignature);

    if (!isValidSignature) {
        return res.status(400).json({ valid: false, error: 'Invalid credential
            signature' });
    }

    const isValid = isValidSignature;
    res.json({ valid: isValid });
} catch (error) {
    console.error('Error verifying credential:', error.message);
    res.status(404).json({ error: error.message });
}
});

app.listen(port, () => {
console.log(Credential verifier running at http://localhost:${port});
});
```

Verifying a Credential To verify a credential, the verifier performs the following steps:

1. Receive Credential: The verifier receives a credential in JSON format from the requester.
2. Query DID Registry: It queries the DID Registry to retrieve the DID document associated with the credential's subject (the DID).
3. Verify Signature: The verifier extracts the public key from the DID document and uses it to verify the digital signature attached to the credential.
4. Respond to Requester: If the signature is valid, the verifier responds with a success message indicating the credential is authentic. If the signature is invalid, it returns an error.

Testing the Credential Verifier Once the Docker containers are running, you can test the credential verification process by sending a POST request to the verifier service at http://localhost:9002/verify-credential. The request should include the credential to be verified in the body of the request.

Example:

```
curl -X POST http://localhost:9002/verify-credential
-H "Content-Type: application/json"
-d '{
"credential": {
"content": {"name": "John Doe", "course": "Blockchain 101"},
"issuedTo": "did:example:1234",
"issuedAt": "2024-08-09T12:34:56.789Z",
"issuer": "issuer1",
"signature": "base64_encoded_signature",
"registry": "db"
}
}'
```

The verifier will check the credential's signature against the public key stored in the DID Registry and return whether the credential is valid.

4.3.7 DID Registry

The DID Registry is a critical component of the Self-Sovereign Identity (SSI) infrastructure. It maintains a registry of Decentralized Identifiers (DIDs) and their corresponding DID Documents, which are essential for resolving identities and verifying credentials within the system. The DID Registry can store DID Documents in both a local JSON database and on the blockchain.

DID Registry Implementation The DID Registry is implemented as an Express.js service that provides endpoints to create, resolve, and delete DIDs and their associated documents. It supports two storage methods: a local JSON database (db.json) and a blockchain-based registry.

Server Setup The index.js file initializes the DID Registry service, sets up the connection to the blockchain, and configures the necessary routes for handling DID operations.

Listing 22: index.js

```
const express = require('express');
const cors = require('cors');
const bodyParser = require('body-parser');
const low = require('lowdb');
const FileSync = require('lowdb/adapters/FileSync');
const Web3 = require('web3');

const app = express();
```

```
const port = 9003;

const abi = [ /* Contract ABI */ ];

const adapter = new FileSync('db.json');
const db = low(adapter);

db.defaults({ dids: [] }).write();

app.use(cors());
app.use(bodyParser.json());

let web3;
let contract;
let account;

const contractAddress = "0xCf7Ed3AccA5a467e9e704C703E8D87F634fB0Fc9";
if (contractAddress) {
web3 = new Web3(new Web3.providers.HttpProvider("http://hardhat-node:8545"));
contract = new web3.eth.Contract(abi, contractAddress);
console.log(Contract loaded from address: ${contractAddress});
}

const setupWeb3AndDeployContract = async () => {
web3 = new Web3(new Web3.providers.HttpProvider("http://hardhat-node:8545"));
account = web3.eth.accounts.privateKeyToAccount('0xac0974bec39a17e36ba4a6b4d238ff944
    bacb478cbed5efcae784d7bf4f2ff80');
web3.eth.accounts.wallet.add(account);
web3.eth.defaultAccount = account.address;
console.log(Account address: ${account.address});
};

setupWeb3AndDeployContract().catch(console.error);

app.use('/dids', (req, res, next) => {
req.db = db;
req.contract = contract;
req.account = account;
next();
}, require('./routes/did'));

app.listen(port, () => {
console.log(DID registry running at http://localhost:${port});
});

module.exports = { contract, account };
```

DID Document Management The did.js file contains the routes and logic for managing DID Documents. It supports the following operations:

- Create a DID Document: Stores a new DID Document either in the JSON database or on the blockchain.
- Resolve a DID Document: Retrieves a DID Document from the JSON database or the blockchain.
- Delete a DID Document: Removes a DID Document from the JSON database or the blockchain.

Listing 23: did.js

```
// Create a new DID Document
router.post('/db', (req, res) => {
const didDocument = req.body;
const db = req.db;

const existingDid = db.get('dids').find({ id: didDocument.id }).value();
if (existingDid) {
return res.status(400).json({ error: 'DID already exists' });
}

db.get('dids').push(didDocument).write();
res.status(201).json(didDocument);
});

// Resolve a DID Document
router.get('/db/
', (req, res) => {
const did = req.params.did;
const db = req.db;

const didDocument = db.get('dids').find({ id: did }).value();
if (!didDocument) {
return res.status(404).json({ error: 'DID not found' });
}

res.json(didDocument);
});

// Delete a DID Document
router.delete('/db/
', (req, res) => {
const did = req.params.did;
const db = req.db;

const didDocument = db.get('dids').find({ id: did }).value();
if (!didDocument) {
return res.status(404).json({ error: 'DID not found' });
}

db.get('dids').remove({ id: did }).write();
res.status(200).json({ message: 'DID deleted successfully' });
});
```

Blockchain Interaction The DID Registry also interacts with a smart contract deployed on the blockchain to manage DIDs. The smart contract allows for storing, retrieving, and deleting DIDs using Ethereum addresses.

Example of storing a DID on the blockchain:

```
// Store a DID on the blockchain
router.post('/blockchain', async (req, res) => {
const didDocument = req.body;
const { contract, account } = req;

const did = didDocument.id;
const address = did.split(':')[2];

if (isValidEthereumAddress(address)) {
```

```
try {
await contract.methods.setDID(address, JSON.stringify(didDocument)).send({ from:
    account.address, gas: 3000000 });
res.status(201).json(didDocument);
} catch (error) {
console.error(Error storing DID on blockchain: ${error.message});
res.status(500).json({ error: 'Error storing DID on blockchain', details: error.
    message });
}
} else {
res.status(400).send({ error: 'Invalid DID format' });
}
});
```

Testing the DID Registry You can test the DID Registry with the testscript

```
node test.js
```

4.4 InterPlanetary File System

This section provides a comprehensive guide to the IPFS (InterPlanetary File System) Docker container, which serves as a decentralized storage infrastructure for other containers within the system. This IPFS node allows the network to store, retrieve, and manage files in a distributed manner, supporting the other services.

4.4.1 Docker Compose Configuration

The `docker-compose.yml` file defines the configuration for the IPFS service. The IPFS container is set up to expose necessary ports for communication, utilize a persistent data volume, and connect to an external network shared by other services.

Listing 24: docker-compose.yml

```
services:
  ipfs:
    image: ipfs/kubo:latest
    container_name: ipfs_node
    ports:
      - "4001:4001"   # swarm
      - "5001:5001"   # API
      - "8080:8080"   # Gateway
    volumes:
      - ipfs_data:/data/ipfs
    networks:
      - hardhat-network

volumes:
  ipfs_data:
    driver: local

networks:
  hardhat-network:
    external: true
```

Explanation of Configuration

- **Services**:
  - **ipfs**: The IPFS service is built from the provided Dockerfile, with its container name set to `ipfs_node`.
  - **Ports**:
    - **4001**: Peer-to-peer (P2P) communication port.
    - **5001**: IPFS API port.
    - **8080**: Gateway port for accessing IPFS-hosted content via HTTP.
  - **Volumes**:
    - **ipfs_data**: Persists the IPFS data, ensuring that files and configurations remain intact across container restarts.
  - **Networks**:
    - **hardhat-network**: The IPFS service is connected to an external network to facilitate interaction with other containers in the SSI infrastructure.

4.4.2 Entrypoint Script

The `entrypoint.sh` script is executed when the container starts. It initializes the IPFS node if necessary, configures it to allow CORS, and starts the IPFS daemon.

Listing 25: entrypoint.sh

```
#!/bin/sh

# Initialize IPFS repository if it doesn't exist
if [ ! -d "/root/.ipfs" ]; then
  ipfs init
fi

# Copy the swarm key to the IPFS configuration directory
cp /data/ipfs/swarm.key /root/.ipfs/swarm.key

# Update IPFS config to listen on all network interfaces
ipfs config --json Addresses.API '["/ip4/0.0.0.0/tcp/5001"]'
ipfs config --json Addresses.Gateway '["/ip4/0.0.0.0/tcp/8080"]'

# Allow CORS for IPFS API
ipfs config --json API.HTTPHeaders.Access-Control-Allow-Origin '["*"]'
ipfs config --json API.HTTPHeaders.Access-Control-Allow-Methods '["GET", "POST", "PUT
    "]'
ipfs config --json API.HTTPHeaders.Access-Control-Allow-Headers '["Authorization"]'

# Start the IPFS daemon
exec ipfs daemon --enable-pubsub-experiment
```

Key Operations in the Entrypoint Script

- **IPFS Initialization**: If the IPFS repository is not already initialized (i.e., the directory `/root/.ipfs` does not exist), the script initializes it using `ipfs init`.
- **Swarm Key Management**: The script copies the swarm key to the IPFS configuration directory, ensuring that the node joins the correct IPFS private network.
- **Configuration Updates**:

  - The IPFS API and Gateway are configured to listen on all network interfaces.
  - CORS headers are set to allow cross-origin requests to the IPFS API.

- **Daemon Startup**: Finally, the IPFS daemon is started with the `--enable-pubsub-experiment` flag to support IPFS's experimental pubsub feature.

4.5 Oracle Infrastructure

The Oracle Docker container is responsible for providing external data to smart contracts on the blockchain, serving as a bridge between off-chain data sources and on-chain smart contracts. It interacts with other components, such as the IPFS and the credential verifier, to fetch, verify, and store data on the blockchain.

4.5.1 Docker Compose Configuration

The `docker-compose.yml` file defines the configuration for the Oracle service. The service is exposed on port 9010 and is connected to the same network as other components like the IPFS and blockchain nodes.

Listing 26: docker-compose.yml

```
services:
oracle:
build: ./oracle
ports:
- "9010:9010"
networks:
- hardhat-network

networks:
hardhat-network:
external: true
```

4.6 User Interface

The user interface is composed of the following HTML pages:

- **counter.html**: A page for interacting with a counter contract deployed on the blockchain.
- **crypto-wallet.html**: A page that serves as a user interface for managing crypto wallets and performing Ether transfers.
- **ERC20.html**: A page dedicated to interacting with an ERC20 token contract.
- **ERC721.html**: A page for creating, viewing, and transferring NFTs using an ERC721 contract.
- **ipfs.html**: A page for uploading and downloading files from IPFS.
- **issue.html**: A page for issuing digital credentials.
- **ssi-wallet.html**: A page that acts as a wallet for managing digital credentials.
- **verify.html**: A page for verifying the authenticity of issued credentials.

4.6.1 JavaScript Functionality

Each HTML page is powered by JavaScript, which handles interactions with the backend services, blockchain, and IPFS. Below is an analysis of the key JavaScript files associated with these pages.

generic-wallet.js This script is used across multiple pages to provide basic wallet functionality. It checks for the availability of MetaMask, loads initial addresses, and allows transactions on the blockchain.

Listing 27: generic-wallet.js

```
let web3;

if (typeof window.ethereum !== 'undefined' && window.ethereum.isMetaMask) {
    web3 = new Web3(new Web3.providers.HttpProvider("http://localhost:8545"));
} else if (typeof window.web3 !== 'undefined') {
    web3 = new Web3(window.web3.currentProvider);
} else {
    web3 = new Web3(new Web3.providers.HttpProvider("http://localhost:8545"));
}

const initialAddresses = [
    { address: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266', privateKey: '...' },
    { address: '0x8626f6940E2eb28930eFb4CeF49B2d1F2C9C1199', privateKey: '...' },
    // Additional addresses...
];

// Function to load initial addresses
function loadInitialAddresses() {
    localStorage.setItem('addresses', JSON.stringify(initialAddresses));
}

// Function to get the balance of an address
async function getBalance(address) {
    try {
        const balance = await web3.eth.getBalance(address);
        return web3.utils.fromWei(balance, 'ether');
    } catch (error) {
        console.error('Error fetching balance:', error);
        throw error;
    }
}

// Load initial addresses on startup
loadInitialAddresses();
```

Purpose: This script initializes the web3 instance, loads initial Ethereum addresses, and provides functions to get the balance of an address and sign transactions. It is essential for any page that interacts with the blockchain.

crypto-wallet.js This script powers the `crypto-wallet.html` page, allowing users to manage their Ethereum addresses, upload smart contracts, and transfer Ether.

Listing 28: crypto-wallet.js

```
document.addEventListener('DOMContentLoaded', (event) => {
    loadInitialAddresses();
    loadAddresses();
    loadContracts();
});

// Function to load addresses and populate the UI
function loadAddresses() {
    const addresses = JSON.parse(localStorage.getItem('addresses')) || [];
    const addressSelect = document.getElementById('address-select');
    addressSelect.innerHTML = '';
```

```
    addresses.forEach((addressObj, index) => {
        const option = document.createElement('option');
        option.value = index;
        option.text = addressObj.address;
        addressSelect.add(option);
    });

    // Load the balance when an address is selected
    addressSelect.addEventListener('change', displayBalance);
}

// Function to delete an address
function deleteAddress() {
    const addressSelect = document.getElementById('address-select');
    const selectedIndex = addressSelect.value;

    let addresses = JSON.parse(localStorage.getItem('addresses')) || [];
    addresses.splice(selectedIndex, 1);
    localStorage.setItem('addresses', JSON.stringify(addresses));

    loadAddresses();
    clearAddressDetails();
}

// Function to transfer Ether
async function transferEther() {
    const fromSelect = document.getElementById('transfer-from');
    const toInput = document.getElementById('transfer-to');
    const amountInput = document.getElementById('transfer-amount');

    const fromIndex = fromSelect.value;
    const toAddress = toInput.value;
    const amount = amountInput.value;

    if (fromIndex === '' || toAddress === '' || amount === '') {
        alert('Please fill in all fields');
        return;
    }

    transferETH(fromIndex, toAddress, amount);
}
```

Purpose: The script manages the crypto wallet functionalities, including loading addresses, displaying balance, deleting addresses, and transferring Ether. It integrates with the Ethereum blockchain using the web3.js library.

ERC20.js This script enables interaction with an ERC20 token smart contract on the `ERC20.html` page. It allows users to view balances, transfer tokens, and approve allowances.

Listing 29: ERC20.js

```
const contractAddress = "0xc351628EB244ec633d5f21fBD6621e1a683B1181"; // Replace with
    actual contract address
const abi = [ /* ERC20 contract ABI */ ];

document.getElementById('transfer').onclick = () => {
    const recipient = document.getElementById('recipient').value;
    const amount = document.getElementById('amount').value;
```

```
    performTransaction('transfer', [recipient, amount]);
};

// Function to update the balance
async function updateBalance() {
    const balanceElement = document.getElementById('balance');
    const balance = await callContractMethod(contractAddress, abi, 'balanceOf', [
        selectedWallet]);
    balanceElement.textContent = ethers.utils.formatUnits(balance, decimals);
}
```

Purpose: This script manages interactions with the ERC20 contract, allowing users to transfer tokens, check balances, and manage approvals.

ERC721.js This script manages NFT creation, metadata viewing, and transfer on the `ERC721.html` page.

Listing 30: ERC721.js

```
const contractAddress = "0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0"; // Replace with
    actual contract address
const abi = [ /* ERC721 contract ABI */ ];

document.getElementById('createNFT').onclick = mintNFT;
document.getElementById('viewMetadata').onclick = viewMetadata;
document.getElementById('transferNFT').onclick = transferNFT;

// Function to mint a new NFT
async function mintNFT() {
    const uri = document.getElementById('nftURL').value;
    try {
        await sendTransaction(contractAddress, abi, 'safeMint', [selectedWallet, uri],
            selectedWallet);
        alert('NFT minted successfully');
    } catch (error) {
        console.error('Error minting NFT:', error);
    }
}
```

Purpose: The script allows users to mint new NFTs, view their metadata, and transfer them to other addresses. It interacts with the ERC721 smart contract.

ipfs.js This script enables file upload and download to and from IPFS on the `ipfs.html` page.

Listing 31: ipfs.js

```
document.getElementById('upload-button').addEventListener('click', uploadFile);
document.getElementById('download-button').addEventListener('click', downloadFile);

// Function to upload a file to IPFS
async function uploadFile() {
    const fileInput = document.getElementById('file-input');
    const file = fileInput.files[0];
    if (!file) {
        alert('Please select a file to upload');
        return;
    }

    const formData = new FormData();
    formData.append('file', file);
```

```
    try {
        const response = await fetch('http://localhost:5001/api/v0/add', {
            method: 'POST',
            body: formData
        });
        const data = await response.json();
        document.getElementById('upload-result').textContent = `File uploaded: ${data.
            Hash}`;
    } catch (error) {
        console.error('Error uploading file:', error);
    }
}
```

Purpose: This script handles file uploads and downloads, interfacing with the IPFS network through the IPFS HTTP client.

issue.js This script powers the credential issuance process on the `issue.html` page.

Listing 32: issue.js

```
const issuerUrl = 'http://localhost:9001/issue-credential';

document.getElementById('issue-credential').addEventListener('click', async () => {
    const registry = document.getElementById('registry').value;
    const issuer = document.getElementById('issuer').value;
    const did = document.getElementById('did').value;
    const name = document.getElementById('name').value;
    const course = document.getElementById('course').value;

    try {
        const response = await fetch(issuerUrl, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
                registry: registry,
                issuer: issuer,
                did: did,
                credential: { type: 'StudentID', name, course }
            })
        });
        const credential = await response.json();
        storeCredential(credential);
        document.getElementById('issued-credential').textContent = JSON.stringify(
            credential, null, 2);
    } catch (error) {
        console.error('Error issuing credential:', error);
    }
});
```

Purpose: The script is responsible for issuing credentials, interacting with the issuer service and storing the issued credential in the browser's local storage.

verify.js This script enables the verification of credentials on the `verify.html` page.

Listing 33: verify.js

```
const verifierUrl = 'http://localhost:9002/verify-credential';

document.getElementById('verify-credential').addEventListener('click', async () => {
```

```
    const did = document.getElementById('verify-did').value;
    const name = document.getElementById('verify-name').value;
    const course = document.getElementById('verify-course').value;
    const issuedAt = document.getElementById('verify-issuedAt').value;

    const storedCredentials = JSON.parse(localStorage.getItem('credentials')) || [];
    const selectedCredential = storedCredentials.find(c => c.issuedTo === did && c.
        issuedAt === issuedAt);

    try {
        const response = await fetch(verifierUrl, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ credential: selectedCredential })
        });
        const result = await response.json();
        document.getElementById('verification-result').textContent = result.valid ? '
            Credential is valid' : 'Credential is invalid';
    } catch (error) {
        console.error('Error verifying credential:', error);
    }
});
```

Purpose: The script allows users to verify credentials by sending them to a verifier service and displaying the results.

ssi-wallet.js This script powers the `ssi-wallet.html` page, which acts as a wallet for managing credentials.

Listing 34: ssi-wallet.js

```
document.addEventListener('DOMContentLoaded', loadCredentials);

function loadCredentials() {
    const credentials = JSON.parse(localStorage.getItem('credentials')) || [];
    const credentialSelect = document.getElementById('credential-select');
    credentialSelect.innerHTML = '';

    credentials.forEach((credential, index) => {
        const option = document.createElement('option');
        option.value = index;
        option.text = `${credential.content.name} - ${credential.issuedTo}`;
        credentialSelect.add(option);
    });

    credentialSelect.addEventListener('change', (event) => {
        const selectedIndex = event.target.value;
        const credential = credentials[selectedIndex];
        document.getElementById('view-did').value = credential.issuedTo;
        document.getElementById('view-name').value = credential.content.name;
        document.getElementById('view-course').value = credential.content.course;
        document.getElementById('view-issuedAt').value = credential.issuedAt;
        document.getElementById('view-registry').value = credential.registry;
    });
}

// Function to delete a credential and its DID
function deleteCredentialAndDid() {
    const credentialSelect = document.getElementById('credential-select');
```

```
    const selectedIndex = credentialSelect.value;

    let credentials = JSON.parse(localStorage.getItem('credentials')) || [];
    const credential = credentials[selectedIndex];

    if (credential) {
        fetch(`http://localhost:9003/dids/${credential.registry}/${credential.issuedTo
            }`, { method: 'DELETE' })
            .then(response => {
                if (response.ok) {
                    credentials.splice(selectedIndex, 1);
                    localStorage.setItem('credentials', JSON.stringify(credentials));
                    loadCredentials();
                    alert('Credential and DID deleted successfully');
                } else {
                    alert('Error deleting DID from registry');
                }
            })
            .catch(error => {
                console.error('Error deleting DID:', error);
            });
    }
}

document.getElementById('delete-credential-did').addEventListener('click',
    deleteCredentialAndDid);
```

Purpose: The script enables users to manage their digital credentials, delete them, and remove associated DIDs from the registry.